

WEEK 13

Q1) Given an array of numbers, find the index of the smallest array element (the pivot), for which the sums of all elements to the left and to the right are equal. The array may not be reordered.

Example

arr=[1,2,3,4,6]

- the sum of the first three elements, $1+2+3=6$. The value of the last element is 6.
- Using zero based indexing, arr[3]=4 is the pivot between the two subarrays.
- The index of the pivot is 3.

Function Description

Complete the function balancedSum in the editor below.

balancedSum has the following parameter(s):

int arr[n]: an array of integers Returns:

int: an integer representing the index of the pivot Constraints

- $3 \leq n \leq 10^5$
- $1 \leq arr[i] \leq 2 \times 10^4$, where $0 \leq i < n$
- It is guaranteed that a solution always exists

Input Format for Custom Testing

Input from stdin will be processed as follows and passed to the function. The first

line contains an integer n, the size of the array arr.

Each of the next n lines contains an integer, arr[i], where $0 \leq i < n$.

Sample Case 0

Sample input 0

STDIN Function Parameters

4 → arr[] size = 4

1 → arr = [1, 2, 3, 3]

2

3

3

SampleOutput0

2

Explanation0

- The sum of the first two elements, $1+2=3$. The value of the last element is 3.
- Using zero based indexing, $arr[2]=3$ is the pivot between the two subarrays.
- The index of the pivot is 2.

SampleCase1

SampleInput1

STDIN FunctionParameters

3 $\rightarrow arr[]$ size = 3

1 $\rightarrow arr = [1, 2, 1]$

2

1

SampleOutput1

1

Explanation1

- The first and last elements are equal to 1.
- Using zero based indexing, $arr[1]=2$ is the pivot between the two subarrays.
- The index of the pivot is 1.

```

1 1 /*
2 2  * Complete the 'balancedSum' function below.
3 3  *
4 4  * The function is expected to return an INTEGER.
5 5  * The function accepts INTEGER_ARRAY arr as parameter.
6 6  */
7 7
8 8 int balancedSum(int arr_count, int* arr)
9 9 {
10 10     int left = 0, right = 0;
11 11     for(int i=0; i<arr_count;i++){
12 12         right += arr[i];
13 13     }
14 14     for(int i=0;i<arr_count;i++){
15 15         if(left==(right-arr[i]))
16 16             return i;
17 17         left += arr[i];
18 18         right -= arr[i];
19 19     }
20 20     return 1;
21 21 }
22 22

```

	Test	Expected	Got	
✓	int arr[] = {1,2,3,3}; printf("%d", balancedSum(4, arr))	2	2	✓

Passed all tests! ✓

Q2) Calculatethesumofanarrayofintegers.

Example

numbers=[3,13,4,11,9]

Thesumis3+13+4+11+9=40. Function

Description

ComplethefunctionarraySumintheeditorbelow. arraySum

has the following parameter(s):

intnumbers[n]:anarrayofintegers

Returns

int:integersumofthenumbersarray

Constraints

$1 \leq n \leq 10^4$

$1 \leq \text{numbers}[i] \leq 10^4$

InputFormatforCustomTesting

Inputfromstdinwillbeprocessedasfollowsandpassedtothefunction. The first line contains an integer n , the size of the array numbers.

Eachofthenextnlinescontainsanintegernumbers[i]where $0 \leq i < n$.

SampleCase0

Sample Input 0

STDIN Function

5 $\rightarrow$ numbers[size=5

1 $\rightarrow$ numbers=[1,2,3,4,5]

2

3

4

5

SampleOutput0

15

Explanation0

$1 + 2 + 3 + 4 + 5 = 15$.

SampleCase1

Sample Input 1

STDIN Function

2 $\rightarrow$ numbers[size=2

12 $\rightarrow$ numbers=[12,12]

12

SampleOutput1

24

Explanation1

12+12= 24.

```

1  /*
2   * Complete the 'arraySum' function below.
3   *
4   * The function is expected to return an INTEGER.
5   * The function accepts INTEGER_ARRAY numbers as parameter.
6   */
7
8  int arraySum(int numbers_count, int *numbers)
9  {
10     int sum=0;
11     for(int i=0;i < numbers_count;i++){
12         sum += numbers[i];
13     }
14     return sum;
15 }
16

```

	Test	Expected	Got	
✓	int arr[] = {1,2,3,4,5}; printf("%d", arraySum(5, arr))	15	15	✓

Passed all tests! ✓

Q3) Given an array of n integers, rearrange them so that the sum of the absolute differences of all adjacent elements is minimized. Then, compute the sum of those absolute differences. Example $n=5$ $arr = [1, 3, 3, 2, 4]$ If the list is rearranged as $arr' = [1, 2, 3, 3, 4]$, the absolute differences are $|1 - 2| = 1$, $|2 - 3| = 1$, $|3 - 3| = 0$, $|3 - 4| = 1$. The sum of those differences is $1 + 1 + 0 + 1 = 3$. Function Description Complete the function `minDiff` in the editor below. `minDiff` has the following parameter: `arr`: an integer array Returns: `int`: the sum of the absolute differences of adjacent elements Constraints $2 \leq n \leq 105$ $0 \leq arr[i] \leq 109$, where $0 \leq i < n$ Input Format For Custom Testing The first line of input contains an integer, n , the size of `arr`. Each of the following n lines contains an integer that describes `arr[i]` (where $0 \leq i < n$).

Sample Case 0 Sample Input For Custom Testing STDIN Function -----5 → `arr` size = 5 → `arr` = [5, 1, 3, 7, 3] 1 3 7 3 Sample Output 6 Explanation $n=5$ $arr = [5, 1, 3, 7, 3]$ If `arr` is rearranged as `arr' = [1, 3, 3, 5, 7]`, the differences are minimized. The final answer is $|1 - 3| + |3 - 3| + |3 - 5| + |5 - 7| = 6$.

Sample Case 1 Sample Input For Custom Testing STDIN Function -----2 → `arr` size = 2 → `arr` = [1, 2]

= [3,2] 2 Sample Output 1 Explanation n=2 arr=[3,2] There is no need to rearrange because there are only two elements. The final answer is $|3 - 2| = 1$.

```

1  /*
2   * Complete the 'minDiff' function below.
3   *
4   * The function is expected to return an INTEGER.
5   * The function accepts INTEGER_ARRAY arr as parameter.
6   */
7
8  int minDiff(int arr_count, int* arr)
9  {
10     for(int i=0; i<arr_count-1;i++){
11         for(int j=0;j<arr_count-i-1;j++){
12             if(arr[j]>arr[j+1]){
13                 int temp = arr[j];
14                 arr[j]=arr[j+1];
15                 arr[j+1]=temp;
16             }
17         }
18     }
19     int sum=0;
20     for(int i=0;i<arr_count-1;i++){
21         sum += abs(arr[i]-arr[i+1]);
22     }
23     return sum;
24 }
25

```

	Test	Expected	Got	
✓	int arr[] = {5, 1, 3, 7, 3}; printf("%d", minDiff(5, arr))	6	6	✓

Passed all tests! ✓